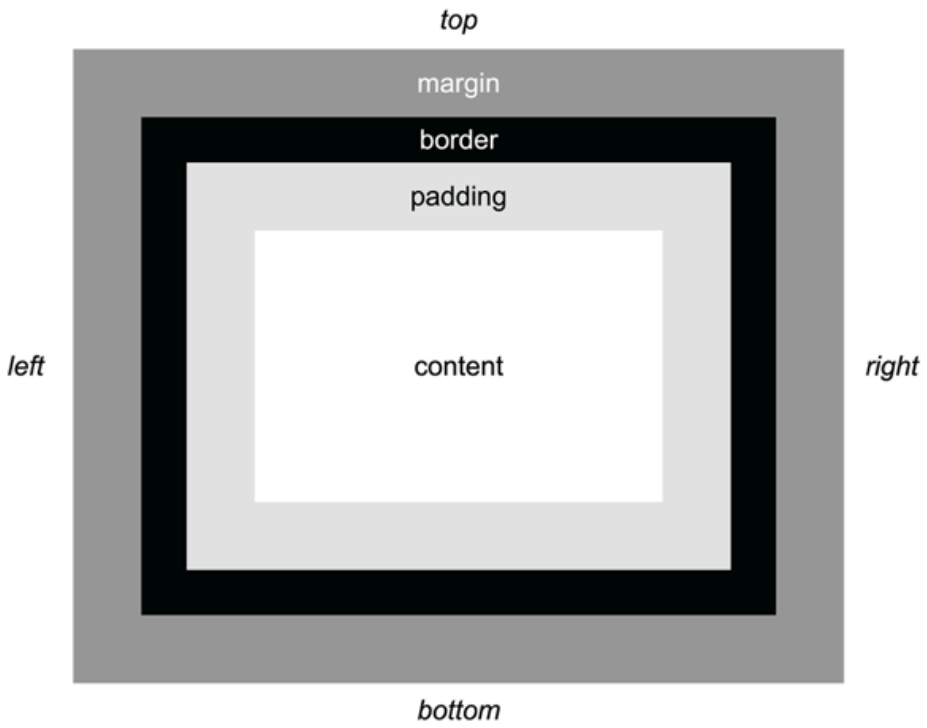
Modul 2 : CSS Basic

Cascading Style Sheets (CSS) are used to define and customize the styles and layouts for your web pages. This means you can create style sheets to alter the design, layout, and responsiveness to different screen sizes on various devices from computers to smartphones.

CSS describes how HTML elements are to be displayed on screen and controls the layout of multiple web pages all at once. This is because the style sheets are stored in separate CSS files and are linked to the HTML document

1. **CSS Box Model**



The components of the CSS box model

- **Content:** The stuff inside the box, such as text, links, lists, images, and video. In other words, the content of all the boxes in your page is nothing more nor less than the actual content you've added to the page. The four sides of the content area — that is, its top, left, bottom, and right edges — define the content box.
- **Padding:** The space that surrounds the content. That is, it's the space just outside the content, but inside the border.
- **Border:** The space that surrounds the padding. That is, it's the space just outside the padding, but inside the margin.
- **Margin:** The space that surrounds the border. That is, it's the space just outside the border

Types of boxes:

- **Block boxes:** This type of box is rendered on the web page using the following characteristics:
    - The box starts on a new line.
    - If you don't set a width for the box, the box will fill the entire horizontal space of its containing element.
    - If you set the width and height of the box, those properties get applied.
    - Increasing the size of the box padding, borders, and margins causes surrounding elements to be pushed away from the box, while decreasing the size of the box padding, borders, and margins causes surrounding elements to be drawn closer to the box.

  By definition, the default value for the display property of a block box is block:

  ```
  display: block;
  ```

- **Inline boxes:** This type of box is rendered on the web page as follows:
    - The box flows along with the surrounding content.
    - The box will only be as wide as its content requires.
    - If you set the width and height of the box, those properties do not get applied.
    - Increasing or decreasing the size of the vertical padding, borders, and margins has no effect on the surrounding elements.
    - Increasing the size of the horizontal padding, borders, and margins causes surrounding elements to be pushed away from the box, while decreasing the size of the horizontal padding, borders, and margins causes surrounding elements to be drawn closer to the box.

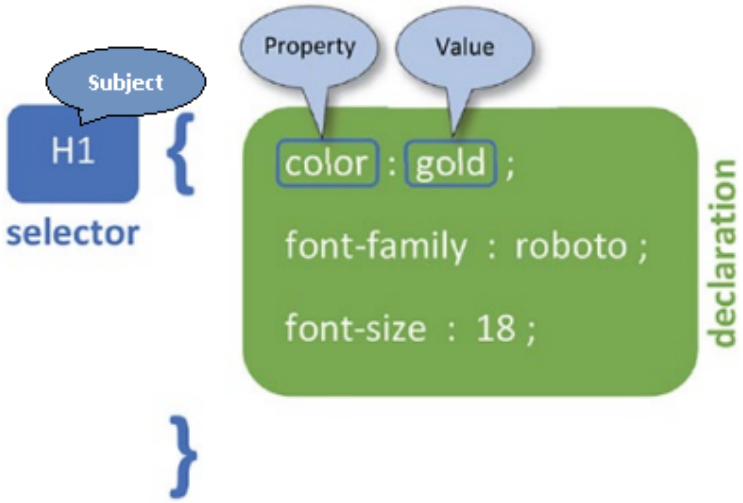
  By definition, the default value for the display property of an inline box is inline:

  ```
  display: inline;
  ```

2. CSS Selector

A selector is a pattern that you use to define, as accurately as possible, which element or elements on the page you want to style.

Any element referenced by a selector is known as the <u>subject</u> of the selector



CSS Syntax

One of the most powerful aspects of selectors is that you can take a single CSS declaration block and apply it to multiple selectors.

```
selectorA, selectorB, …, selectorN {
one or more CSS declarations go here
}
```

For easier reading, most web developers put each selector on its own line, like so:

```
selectorA,
selectorB,
…,
selectorN {
one or more CSS declarations go here
}
```

Standard Selectors

The type selector	`element {`
	` property1: value1;`
	` property2: value2;`
	` …`
	`}`
	For example
	`aside {`
	` border: 3px solid black;`
	`}`

The class selector (.)	`.element {`
	` property1: value1;`
	` property2: value2;`
	` …`
	`}`
	For example
	`<h2 class="subtitle">`
	`----------`
	`.subtitle {`
	`  color: royalblue;`
	`  font-style: italic;`
	`}`
	
	Combining <u>type and class</u> selectors
	`element.class-name {`
	` property1: value1;`
	` …`
	`}`
	
	Matching <u>multiple</u> classes
	It's possible (in fact, fairly common) for an element to have two or more classes.
	
	`<h4 class="subtitle aside-title">`
	
	`.subtitle{`
	`  font-style: italic;`
	`  }`
	
	`.aside-title{`
	`  color: royalblue;`
	`  }`

The id selector (#)	`#element {`
	` property1: value1;`
	` property2: value2;`
	` …`
	`}`
	For example
	`<h2 class="subtitle">`
	`----------`

	`.subtitle {`
	`  color: royalblue;`
	`  font-style: italic;`
	`}`

The universal selector (*)	`* {`
	`  property1: value1;`
	`  property2: value2;`
	`  …`
	`}`
	
	Some web developers use the universal selector to perform a simple reset of their CSS:
	
	`* {`
	`  padding: 0;`
	`  margin: 0;`
	`  box-sizing: border-box;`
	`}`

Selecting Descendants, Children, and Siblings

One of the most powerful and useful categories of CSS selectors is the collection of so-called combinators. These are operators (such as > and ~) that enable you to combine other types of selectors in a way that matches a specific type of relationship between the selectors: ***descendant, child, or sibling***.

1. **The descendant combinator ( )**

   Rather than targeting specific tags, classes, or ids, you may need to target every instance of a particular element that is contained within another element. I'm talking here not just about the children of some parent element, but also the descendants of that element.

   To apply styles to all of an ancestor element's descendants, CSS offers the *descendant combinator*. To set up a selector that uses a descendant combinator, you include in your rule the ancestor and the descendant type you want to style, separated by a space:

```
ancestor descendant {
 property1: value1;
 property2: value2;
 …
}
```

For example, consider the following HTML

```
<aside>
        <h4>Links:</h4>
        <div>
              <a href="example01.html">The type selector</a><br>
              <a href="example02.html">The class selector</a><br>
              <a href="example03.html">The id selector</a>
        </div>
</aside>
```

Here's a rule that applies a few styles to the <a> tags that are descendants of the <aside> tag:

```
aside a {
 color: red;
 font-style: italic;
 text-decoration: none;
}
```

2. **The child combinator (>)**

To target only those descendants that reside <u>one level down</u>: in short, the children of some parent element.

To aim some styles at the child elements of a parent, you use the child combinator, where you separate the parent and child elements with a greater-than sign (>):

```
parent > child {
 property1: value1;
 property2: value2;
 …
}
```

For example, here's a rule that targets any `h4` element that's a child of an `aside` element

```
aside > h4 {
 color: green;
 border-top: 3px solid black;
 border-bottom: 5px double black;
 padding: 4px 0;
 text-transform: uppercase;
}
```

3. **The subsequent-sibling combinator (~)**

   One common CSS task is to apply a style rule to a particular subject that meets the following criteria:

   - The target element appears in the HTML after a specified element, which is known as the reference element.
   - The target element and the reference element are siblings.

   To apply some styles to such a subject, you use the subsequent-sibling combinator, where you separate the reference and target elements with a tilde (~):

```
reference ~ target {
 property1: value1;
 property2: value2;
 …
}
```

   For example, here's a rule that targets any `ul` element that's a subsequent sibling of an `h2` element

```
h2 ~ ul {
 background: lightpink;
 border: 5px outset crimson;
 list-style-type: square;
 padding: 8px 20px;
}
```

4. **The next-sibling combinator (+)**

   To target only the next sibling that comes after the reference element use the next-sibling combinator, where you separate the reference and target elements with a plus sign (+):

```
reference + target {
 property1: value1;
 property2: value2;
 …
}
```

   For example, here's a rule that targets any `p` element that's the next sibling of an `h2` element

```
h2 + p {
 font-style: italic;
}
```

Selecting Elements by Attribute

An attribute, as I discuss in Book 1, Chapter 1, is some extra code inserted into a tag that modifies the tag in some way. In an <a> tag, for example, the href attribute is set to a string that specifies the address of the link.

Attributes also enable you to differentiate elements in your page. For example, your page may have a bunch of a elements, but only one a element whose href attribute equals https://www.w3.org/.

CSS takes advantage of this differentiability by offering several attribute selectors that enable you to target the presence of a specified attribute, the value of a specified attribute, or some subset of that value. Here's the general syntax for an attribute selector:

```
element[expression] {
 property1: value1;
 property2: value2;
 …
}
```

The expression part of the syntax is the key here, and it's usually a combination of an attribute name, an operator, and a value. Table below takes you through the available expression types.

Selector Name	Syntax	What It Matches

Attribute only	`element[attr]`	Any instance of `element` that includes the attribute named `attr`.
		
		The following example styles all abbr elements that have the title attribute
		
		`abbr[title] {`
		`border: 1px dotted gray;`
		`}`

Attribute equals	`element[attr="value"]`	Any instance of element that includes the attribute named attr
		
		The following example styles all a elements that have an href attribute equal to https://www.w3.org/
		
		`a[href="https://www.w3.org/"] {`
		`color: green;`
		`}`

Attribute begins wth	`element[attr^="value"]`	Any instance of element that includes the attribute named attr
		
		The following example styles all a elements that have an href attribute that begins with http://
		
		`a[href^="http://"] {`
		`color: red;`
		`}`

Attribute contains	`element[attr*="value"]`	Any instance of element with the attribute named attr, the value of which includes value.
		
		The following example styles all div elements that have an id attribute that includes the string navbar
		
		`div[id*="navbar"] {`
		`font-size: 14px;`
		`}`

Attribute ends with	`element[attr$="value"]`	Any instance of element with the attribute named attr, the value of which ends with value
		
		The following example styles all a elements that have an href attribute that ends with .org
		
		`a[href$=".org"] {`
		`color: firebrick;`
		`}`

Attribute is first	`element[attr|="value"]`	Any instance of element with the attribute name attr, the value of which is either equal to value or starts with value- (that is, value followed by a hyphen [-]).
		
		The following example matches all div elements that have a lang attribute that equals en or starts with en-
		
		`div[lang|="en"] {`
		`color: blue;`
		`}`

Attribute includes	`element[attr~="value"]`	Any instance of element with the attribute named attr, the value of which includes value as a whole Word.
		
		The following example matches all img elements that have an alt attribute that includes photo as a whole word
		
		`img[alt~="photo"] {`
		`border: 12px ridge saddlebrown;`
		`}`
		
		Note that this selector would not match alt attributes that include words such as photograph or telephoto

3. CSS Pseudo-Classes & Pseudo-Elements

Your page has a hidden world that's not evident (at least not explicitly) in its HTML code. Has a page visitor selected a checkbox or radio button? Is a user hovering the mouse pointer over an image? Has the user visited one of your links in the past? Is a particular element the first child of its parent element?

A pseudo-class is a CSS selector that acts like a class by generically targeting elements that meet some condition. All pseudo-classes begin with a colon (:),

followed by one or more dash-separated words. You can use a pseudo-class on its own or modified by an element. Here's the general on-its-own syntax:

```
:pseudo-class {
 property1: value1;
 property2: value2;
 …
}
```

Pseudo-classes	What It Matches	Example
Child Elements		
:first-child	The :first-child pseudo-class targets any child element that is the first of a parent element's children	All paragraphs, that is, except the first one, which should have no indent `p:first-child { text-indent: 0; }`
:last-child	The :last-child pseudo-class targets any child element that is the last of a parent element's children	To add some extra whitespace to the bottom of the last paragraph before a heading `p:last-child { margin-bottom: 24px; }`
:nth-child()	The :nth-child() pseudo-class selects one or more elements based on their position in a parent element's collection of siblings.	Here's a selector that targets just the even tr elements of a parent table Element `tr:nth-child(even) { background-color: lightgray; }`
:nth-last-child()	The :nth-last-child() pseudo-class selects one or more elements based on their position in a parent element's collection of siblings, counting from the end.	`element:nth-last-child(n [of list]) { property1: value1; property2: value2; …`

		`}`
:only-child	The :only-child pseudo-class targets any element that’s the only child of a parent elemen	For example, here’s a rule that styles any section element descendant that’s an only child `section *:only-child { color: plum; }`
Type of Elements		
:first-of-type	The :first-of-type pseudo-class targets any child element that's the first of its type in a parent element’s children	`element:first-of-type { property1: value1; property2: value2; …}`
:last-of-type	The :last-of-type pseudo-class targets any child element that's the last of its type in a parent element’s children:	targets the last p element of its type in any parent element `p:last-of-type { margin-bottom: 24px; }`
:nth-of-type()	The :nth-of-type() pseudo-class selects one or more elements of a specified type based on their position in a parent element's collection of siblings	Here's a selector that targets every third p element `p:nth-of-type(3n) { background-color: gray; }`
:nth-last-of-type()	:nth-last-of-type()	`element:nth-last-of-type(n) { property1: value1; property2: value2; … }`
:only-of-type	:only-of-type	For example, here’s a rule that styles any article element div descendant

		that's an only child `article div:only-of-type { margin: 24px 8px; }`
State of Elements		

:checked	Any instance of a checkbox, radio button,
	or selection list option that's toggled on
		input[type="checkbox"]:checked
		{
		box-shadow: 0 0 10px 3px red;
		}
		
		input[type="checkbox"]:checked
		+ label {
		color: red;
		}

:default	Any instance of a form element that is in a
	default state, including a checkbox or
	radio button with the checked attribute, a
	selection list option with the selected
	attribute, or a button element with its type
	attribute set to submit
		input[type="radio"]:default
		{
		box-shadow: 0 0 5px 1px forestgreen;
		}
		
		input[type="radio"]:default
		+
		label {
		color: forestgreen;
		}

:disabled	Any form element that has the disabled
	attribute set
		button:disabled {
		cursor: not-allowed;
		opacity: .5;
		}

:enabled	Any form element that doesn't have the
	disabled attribute set
		button:enabled {
		box-shadow: 0 0 5px 3px seagreen;
		color: seagreen;
		}

:in-range	Any form element with a current value
	that's within the values set by the element’s min and max attributes
		input[type="range"]:in-range {
		background-color: lawngreen;
		}

:invalid	Any form element with a current value that	input[type="email"]:invalid {

	doesn't validate, such as a number field with a text value or an email field with an invalid email address (for example, missing the @ sign)	background: lavenderblush; }

:optional	Any form element with a current value
	that's not within the values set by the
	element’s min and max attributes
		input[type="text"]:optional
		{
		background-color:
		lightgoldenrodyellow;
		}

:out-of-range	Any form element with a current value
	that's not within the values set by the
	element’s min and max attributes
		input[type="range"]:out-of
		range {
		background-color:
		tomato;
		}

:read-only	Any form element that has the readonly
	attribute set
		textarea:read-only {
		border: 0;
		resize: none;
		}

:read-write	Any element that the user can edit, which
	could be a form element that's editable by
	default that doesn’t have the readonly
	attribute set, or any element that’s not
	editable by default that has the contenteditable attribute set (check out
	the sidebar “Editing non-editable content”
	that appears right after this table)
		span:read-write {
		background: honeydew;
		caret-color: crimson;
		}

:required	Any form element that has the required
	attribute set
		input[type="text"]:required
		{
		background-color: yellow;
		}

:valid	Any form element with a current value that
	validates, such as a number field with a
	numeric value or an email field with an
	email address
		input[type="email"]:valid {
		background: palegreen;
		}

4. **Measurement Units**

To control the size of certain objects such as fonts or images, the sizes are specified using various units. There are two types: absolute and relative. Units that are absolute are the same size regardless of the parent element or window size. Units that are relative scale in relation to the parent element or window size depending on the unit used. Here are some examples:

Unit	**Meaning**
in	Inch
mm	Milimeter
px	Short for pixels and is usually used to measure the dimensions of an image
pt	Short for points and is used to measure the size of a font
em	Relative to the current font size of the element. If a font is 12pt, each em unit would be 12pt, so 2em would be 12 x 2 which is 24pt, similarly 1.5em would be 18pt
%	Relative to the parent element or window size

CSS @media Rule

The @media rule is used in media queries to apply different styles for different media types/devices.

Media queries can be used to check many things, such as:

- width and height of the viewport
- width and height of the device
- orientation (is the tablet/phone in landscape or portrait mode?)
- resolution

Using media queries are a popular technique for delivering a tailored style sheet (responsive web design) to desktops, laptops, tablets, and mobile phones.

```
@media not|only mediatype and (mediafeature and|or|not mediafeature)
{
  CSS-Code;
}
```

meaning of the not, only and and keywords:

- **`not`**: The not keyword inverts the meaning of an entire media query.
- **`only`**: The only keyword prevents older browsers that do not support media queries with media features from applying the specified styles. It has no effect on modern browsers.

- **`and`**: The and keyword combines a media feature with a media type or other media features.
- They are all optional. However, if you use **`not`** or **`only`**, you must also specify a media type.

You can also have different stylesheets for different media, like this:

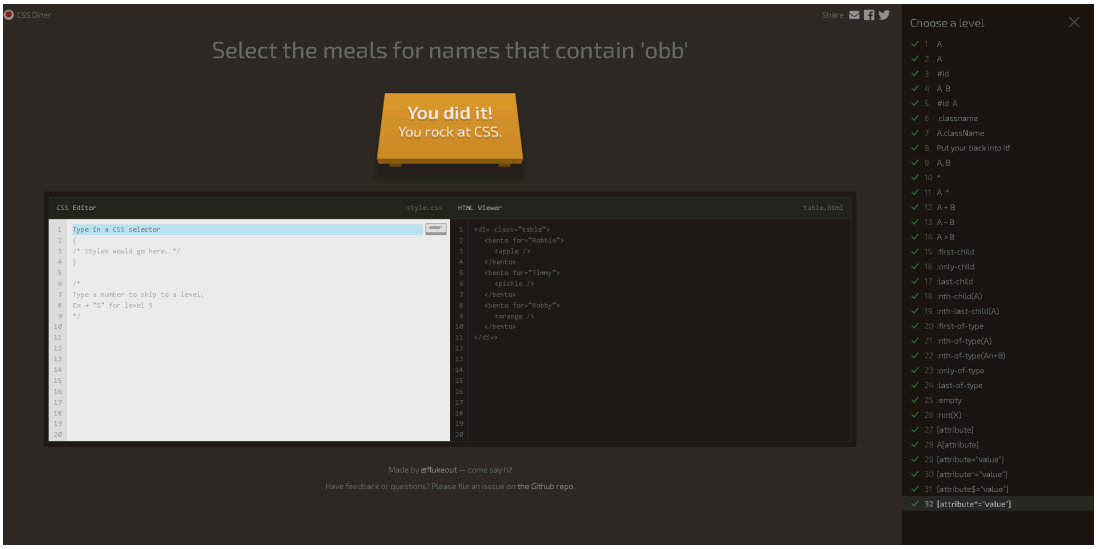
```
<link rel="stylesheet" media="screen and (min-width: 900px)" href="widescreen.css">

<link rel="stylesheet" media="screen and (max-width: 600px)" href="smallscreen.css">
```

CSS Selector Basics Exercise with CSS Diner Game

Instructions:

1. Go to CSS Diner website.
2. Complete the challenges provided in the game by selecting the correct CSS selector to target specific elements.
3. Take the final screenshots of completed stage as PDF document named "**NIM_Name_Module2.pdf**"



Screenshot of completion

4. Submit the PDF document as your assignment.

Note:

1. Ensure that each screenshot clearly shows the completed challenge and the corresponding CSS selector used.
2. Pay attention to the instructions provided in each challenge to select the correct elements.
3. Take your time to understand how CSS selectors work and how they can be applied to target specific elements on a webpage.

This exercise will help you practise and reinforce your understanding of CSS selectors by applying them in a practical game-based scenario.